LIMKOKWING UNIVERSITY OF CREATIVE TECHNOLOGY
SIERRA LEONE

Faculty of INFORMATION COMMUNICATION AND TECHNOLOGY

DCOMP 114 – DATABASE SYSTEMS

ASSIGNMENT TITLE: **Individual Assignment**
LECTURER'S NAME: Sheik Umar Kanu
Issued Date: Week 4
Submission Date: Week 7
NAME OF STUDENT: Marcushia Nancila Grace Kati Marcus-Bangura
STUDENT ID NO: 905005985
CLASS: BSEM1102
SEMESTER: 3/Year 2

Academic Honesty Policy Statement

I hereby attest that the contents of this attachment are my work. Referenced works, articles, art, programs, papers or parts thereof are acknowledged at the end of this paper. Or parts thereof are acknowledged at the end of this paper. This includes data excreted from CD- ROM, the Internet, other private networks, and other people's dicks of the computer system

STUDENT SIGNATURE: Date: 6th/11/2026

MENTOR'S COMMENTS/GRADE:	for office use only upon *receiving* Remark DATE : TIME : RECEIVER'S NAME:

Contents

DOCUMENT GUIDE....3
Suggested contents....3
1. EXECUTIVE SUMMARY....4
2. INTRODUCTION AND BACKGROUND....5
2.1 Introduction....5
2.2 Background of the Study....7
3. AIM, OBJECTIVES AND SCOPE....10
3.1 Aim....10
3.2 Objectives....10
3.3 Scope....10
4. SYSTEM REQUIREMENTS AND TOOLS....11
4.1 Database connection....11
5. DATABASE DESIGN AND STRUCTURE....12
5.1 Main database tables....12
5.2 Key relationships....12
6. TABLE CREATION AND SAMPLE DATA....13
6.1 Data entry and verification....13
7. ADVANCED SQL QUERIES AND JOIN OPERATIONS....14
8. POSTGRESQL VIEWS....22
9. POSTGRESQL TRIGGERS....28
9.1 Trigger One – Automatic enrollment status update....28
9.2 Trigger Two – Examination result audit trail....29
10. STORED PROCEDURE....31
10.1 Procedure operation....31
10.2 Procedure testing....31
11. TESTING AND VERIFICATION....33
12. SECURITY, DATA INTEGRITY AND LIMITATIONS....34
12.1 Data integrity....34
12.2 Auditability....34
12.3 Limitations and future improvements....34
13. CONCLUSION AND RECOMMENDATIONS....35

DOCUMENT GUIDE

This report includes selected screenshot spaces for the principal SQL implementation and demonstration sections. Insert only the relevant pgAdmin screenshots in those spaces.

Suggested contents

- 1. Executive Summary
- 2. Introduction and Background
- 3. Aim, Objectives and Scope
- 4. System Requirements and Tools
- 5. Database Design and Structure
- 6. Table Creation and Sample Data
- 7. Advanced SQL Queries and JOIN Operations
- 8. PostgreSQL Views
- 9. PostgreSQL Triggers
- 10. Stored Procedure
- 11. Testing and Verification
- 12. Security, Integrity and Limitations
- 13. Conclusion and Recommendations
- Appendix A – Screenshot Register
- Appendix B – SQL Source Code Submission

1. EXECUTIVE SUMMARY

EduResults is a relational database project designed to support the management of student information, academic programmes, course registration, academic sessions and examination results. It was implemented using PostgreSQL 18 and managed through pgAdmin 4. The database brings related academic information into a structured system so that authorised users can retrieve student records, review course registrations, analyse examination performance and maintain a record of changes to results.

The implementation includes eight core academic tables—students, departments, programmes, lecturers, courses, academic_sessions, enrollments and results—and an additional result_audit table used by the audit trigger. The project also demonstrates advanced SQL queries and JOINs, five database Views, two Triggers and a Stored Procedure for course registration. The stored procedure was executed and the resulting enrollment for the test student Mohamed Jalloh was verified in the database.

This report describes the database structure, explains the purpose of its major components and provides spaces for inserting the screenshots captured during implementation and testing. The screenshots should be inserted from the original image files; the two supplied photographs show the available screenshot filenames rather than the full-resolution individual pgAdmin captures.

2. INTRODUCTION AND BACKGROUND

2.1 Introduction

The rapid advancement of Information and Communication Technology (ICT) has significantly transformed the way organisations collect, process, store, manage and retrieve information. In the education sector, technology has become an increasingly important tool for improving administrative efficiency, academic management, information accessibility and institutional decision-making. Educational institutions generate and manage large volumes of information daily, including student admission records, personal details, academic programmes, course registrations, lecturer information, examination scores, grades and academic performance reports. Managing these records effectively requires a reliable, organised and efficient information management system.

Traditionally, many educational institutions have depended on manual record-keeping methods, paper-based documentation and separate electronic spreadsheets to manage student information and academic results. Although these methods may serve basic administrative purposes, they often present significant challenges when institutions experience an increase in student enrolment and academic activities. Manual systems can result in misplaced records, duplication of information, delays in retrieving student data, calculation errors, difficulties in updating examination results and challenges in producing accurate academic reports. Furthermore, maintaining the consistency and security of academic records becomes increasingly difficult when information is stored across different files and departments without a centralised management structure.

The need to address these challenges has created an increasing demand for digital information management systems capable of providing structured, reliable and accessible academic data. Database Management Systems (DBMS) offer an effective solution by allowing institutions to store, organise, manipulate and retrieve information through structured database environments. A relational database, in particular, provides a systematic approach to managing information by organising data into interconnected tables. Through the use of primary keys, foreign keys, constraints and relationships, a relational database ensures that information is properly structured and that relationships between different academic records are maintained.

It is against this background that the EduResults – Student Digital Examination and Result Management System was conceptualised and developed. EduResults is a database-driven project designed to demonstrate how database technology can be applied to the management of student academic information, course registration, examination records and academic performance. The project provides a structured database environment in which student records, academic departments, programmes, lecturers, courses, academic sessions, enrollments and examination results can be stored and managed in an organised manner.

The system was developed using PostgreSQL 18 as its relational Database Management System, while pgAdmin 4 was used as the graphical administration and development environment. SQL (Structured Query Language) was employed to create database structures, insert sample records, retrieve information, perform data manipulation and develop advanced database objects. In addition, PostgreSQL

procedural capabilities were used to implement automated database operations through Trigger Functions and Stored Procedures.

EduResults goes beyond the basic creation of database tables. It demonstrates the practical application of relational database concepts through different SQL operations, including INNER JOIN, LEFT JOIN and RIGHT JOIN. These operations allow information stored in different tables to be combined to produce meaningful academic reports. Aggregate functions and analytical queries are also used to examine student performance, calculate average scores, determine grade distributions and organise academic information for reporting purposes.

Furthermore, the project incorporates five database Views to simplify the retrieval of frequently required academic information. These Views provide structured access to student results, course performance, student enrollment summaries, lecturer-course assignments and academic grade distributions. The project also implements two database Triggers to automate selected operations. One Trigger updates the enrollment status when a student's result reaches the published stage, while the second maintains an audit trail of changes made to examination results. A Stored Procedure is also developed to automate student course registration and reduce the possibility of duplicate enrollment records.

Through these features, EduResults demonstrates how a well-designed relational database can improve the organisation, consistency, retrieval and management of academic information. Although the current implementation focuses primarily on the database layer, its structure provides a foundation for future development into a broader digital student information and examination management application.

2.2 Background of the Study

The management of student academic information is one of the fundamental administrative responsibilities of educational institutions. From the point of admission to the completion of an academic programme, institutions are required to maintain accurate records of students, their academic activities, course registrations, examination performances and progression. These records are essential not only for administrative purposes but also for academic planning, institutional monitoring, performance evaluation and decision-making.

As student populations grow and academic programmes become more diverse, the volume of information that institutions must manage also increases. An institution may have several departments, numerous academic programmes, different courses, multiple lecturers and hundreds or thousands of students enrolled across different academic sessions. Each student may register for several courses during an academic semester and receive different assessment and examination scores. Managing these interconnected records requires an information system capable of maintaining accurate relationships between students, courses, academic sessions and examination results.

One of the major challenges associated with traditional record-management approaches is data redundancy. Data redundancy occurs when the same information is unnecessarily stored in multiple locations. For example, a student's personal information may be repeatedly entered into different course registration sheets, examination records and departmental reports. This duplication consumes storage space and increases the likelihood of inconsistencies. If a student's personal details are changed in one record but not updated in another, the institution may end up maintaining conflicting information about the same individual.

Another important challenge is data integrity. Academic records must remain accurate, complete and consistent throughout their lifecycle. Examination scores, grades and enrollment information must be associated with the correct students and courses. Incorrectly entered or improperly updated information can affect student performance reports, academic progression and institutional decisions. A properly structured relational database addresses these concerns through primary keys, foreign keys, unique constraints and other validation mechanisms that help maintain consistency and reliability.

Information retrieval is also a significant consideration in academic administration. Administrators, lecturers and examination officers frequently require information such as individual student results, course enrollment lists, average examination performance, grade distributions and academic summaries. In a poorly organised record-management environment, retrieving this information may require searching through multiple files and manually combining records. A relational database simplifies this process by allowing users to execute SQL queries that retrieve related information from different tables through JOIN operations.

The development of EduResults is therefore based on the practical need to demonstrate how a structured database can address important information-management challenges within an educational environment. The project uses a relational database architecture to organise academic information into separate but interconnected entities. Rather than storing all information in one large table, the database

separates student details, departments, programmes, lecturers, courses, academic sessions, enrollments and examination results into their respective tables. These tables are connected through defined relationships, allowing information to be retrieved without unnecessarily repeating the same data.

The project also recognises the importance of automated database operations. In modern database applications, some activities should occur automatically whenever particular changes are made to stored records. For instance, when an examination result is published, the corresponding enrollment record may need to reflect that the student's academic activity has been completed. Similarly, when examination scores are modified, it is important to maintain a record of the previous and updated values. EduResults addresses these requirements through PostgreSQL Triggers and Trigger Functions.

The audit functionality is particularly important because examination results are sensitive academic records that may influence students' academic progression and final qualifications. Maintaining an audit trail provides a means of identifying when changes occur and examining the previous and updated values recorded by the database. This supports accountability, transparency and traceability in the management of examination information.

In addition to automated operations, the project incorporates Stored Procedures to simplify repetitive database activities. The student registration procedure developed for EduResults accepts relevant student, course and academic-session information and processes the enrollment operation within the database. This reduces the need to repeatedly write lengthy SQL statements for individual registrations and provides a reusable mechanism for carrying out the registration process.

The development of database Views further strengthens the reporting capabilities of EduResults. Views allow frequently used SQL queries to be saved as named database objects, making it easier to retrieve information without rewriting complex JOIN operations each time. Through the five Views developed in this project, different categories of academic information can be presented in a structured and convenient format. These include student examination summaries, course performance reports, enrollment summaries, lecturer-course assignments and academic grade distributions.

From an academic and practical perspective, EduResults also provides an opportunity to demonstrate the application of database design principles and SQL programming concepts in solving a real-world administrative problem. The project brings together several important areas of database development, including relational modelling, data definition, data manipulation, data retrieval, referential integrity, query optimisation concepts, database automation and procedural programming.

The project is particularly relevant to educational institutions seeking to improve the organisation and accessibility of academic information. In the context of Sierra Leone's growing interest in digital transformation and improved educational administration, the development of database-driven solutions presents an opportunity to strengthen institutional record management and support more efficient academic processes. A properly implemented digital database can provide a foundation for improved information access, more reliable reporting and better coordination of academic records.

However, it is important to recognise that EduResults, in its current implementation, represents the database foundation of a proposed student examination and result management system rather than a fully deployed institutional application. Its present functionality is concentrated on database creation, record storage, retrieval, reporting and automation using PostgreSQL. Future development could introduce a web-based interface, secure user authentication, role-based access, online result publication, transcript generation and additional administrative modules.

In conclusion, the background to EduResults is rooted in the need for an organised, reliable and efficient approach to managing student academic information. By applying relational database principles and PostgreSQL programming capabilities, the project demonstrates how technology can support the management of student records, course enrollment and examination results while promoting data consistency, automation, accountability and efficient information retrieval.

3. AIM, OBJECTIVES AND SCOPE

3.1 Aim

The aim of EduResults is to design and implement a structured relational database that supports the storage, retrieval, registration and analysis of student academic information and examination results.

3.2 Objectives

- Create a PostgreSQL database for academic and student-result records.
- Organise data into related tables to reduce unnecessary duplication.
- Use primary keys and foreign keys to connect academic records.
- Insert sample records to demonstrate the database operations.
- Use SQL JOINs and aggregate queries to retrieve and analyse related information.
- Create reusable Views for student, course, lecturer and grade reporting.
- Use Triggers to automate an enrollment-status update and maintain an audit trail of result changes.
- Create and test a Stored Procedure that registers a student for a course.
- Verify the main database objects and document the implementation.

3.3 Scope

The current project focuses on the database layer. It covers student and academic records, course enrollments, examination results, reporting queries, database automation and procedure-based registration. A full web or mobile user interface, authentication portal, fee-payment module, transcript-printing workflow and live integration with an institution's student information system are outside the scope of the database work documented here unless separately developed.

4. SYSTEM REQUIREMENTS AND TOOLS

Tool / component	Use in the project
PostgreSQL 18	Relational database management system used to store data and execute SQL.
pgAdmin 4	Graphical administration tool used to connect to PostgreSQL, create objects and run queries.
SQL	Language used to define tables, insert data, query records and create database objects.
PL/pgSQL	PostgreSQL procedural language used to write Trigger Functions and the Stored Procedure.
Microsoft Word	Recommended application for preparing and formatting this technical report.

4.1 Database connection

The database was accessed in pgAdmin 4 through the PostgreSQL 18 server. The Query Tool was opened with eduresults_db selected as the active database. Running project SQL in the correct database is important because objects created in a different database will not appear in eduresults_db.

5. DATABASE DESIGN AND STRUCTURE

EduResults uses a relational structure. Each core table represents a type of academic entity. Primary keys uniquely identify records, while foreign keys connect records across tables. For example, an enrollment connects a student, a course and an academic session. A result is associated with an enrollment so that the examination record can be traced to the relevant student and course.

5.1 Main database tables

Table	Purpose
departments	Stores academic department records.
programmes	Stores programmes offered by departments.
students	Stores student identity and programme information.
lecturers	Stores lecturer or staff information and department association.
courses	Stores course details and relevant programme/lecturer associations.
academic_sessions	Stores academic-year and semester information.
enrollments	Connects students to courses within academic sessions and records enrollment status.
results	Stores assessment, examination, total-score, grade and result-status information for enrollments.
result_audit	Stores a history of result insertions and updates recorded by the audit Trigger.

5.2 Key relationships

- A department can be associated with multiple programmes and lecturers.
- A programme can be associated with multiple students and courses.
- A student can have multiple enrollment records.
- A course can have multiple enrollment records.
- An academic session can contain many enrollments.
- An enrollment can be associated with an examination result.
- A result can generate audit records when it is inserted or updated.

The exact foreign-key constraints and cardinalities should be confirmed against the final SQL script or pgAdmin ERD. The list above describes the relationships represented by the SQL used in this project.

6. TABLE CREATION AND SAMPLE DATA

The core tables were created in PostgreSQL and populated with sample academic records. Sample data makes it possible to demonstrate joins, result reports, course summaries, trigger behavior and procedure-based registration without using real student records. The project also created the result_audit table to hold the audit information generated during result changes.

6.1 Data entry and verification

INSERT statements were used to add sample records. SELECT statements were then used to inspect the records and confirm that the tables contained the expected information. The additional test student, Mohamed Jalloh (student number LUCT2026007), was used to demonstrate the Stored Procedure registration workflow.

7. ADVANCED SQL QUERIES AND JOIN OPERATIONS

SQL JOIN operations retrieve related information from multiple tables. They are important in EduResults because student details, programme information, course registration and examination results are stored in separate but related tables.

Query / operation	Purpose and interpretation
INNER JOIN – student programme details	Returns students whose programme and department records match the join conditions.
INNER JOIN – student examination results	Combines student, enrollment, course and result information into an examination report.
LEFT JOIN – students and enrollments	Returns every student from the left-side students table, including students without a matching enrollment.
RIGHT JOIN – course enrollment summary	Returns every course from the right-side courses table, including courses with no matching enrollment.
Average performance per course	Uses aggregate functions such as COUNT, AVG, MAX and MIN to summarise course results.
Student performance ranking	Uses a window function such as RANK() to order students by average performance.
Excellent-result filter	Uses WHERE and CASE expressions to identify and categorise high scores.
Grade distribution	Groups results by grade and calculates counts and percentages.

INNER JOIN returns rows with matching values in both joined sources. LEFT JOIN preserves all rows from the left source, while RIGHT JOIN preserves all rows from the right source. Aggregate functions summarise multiple records, and window functions calculate rankings while retaining individual grouped rows.

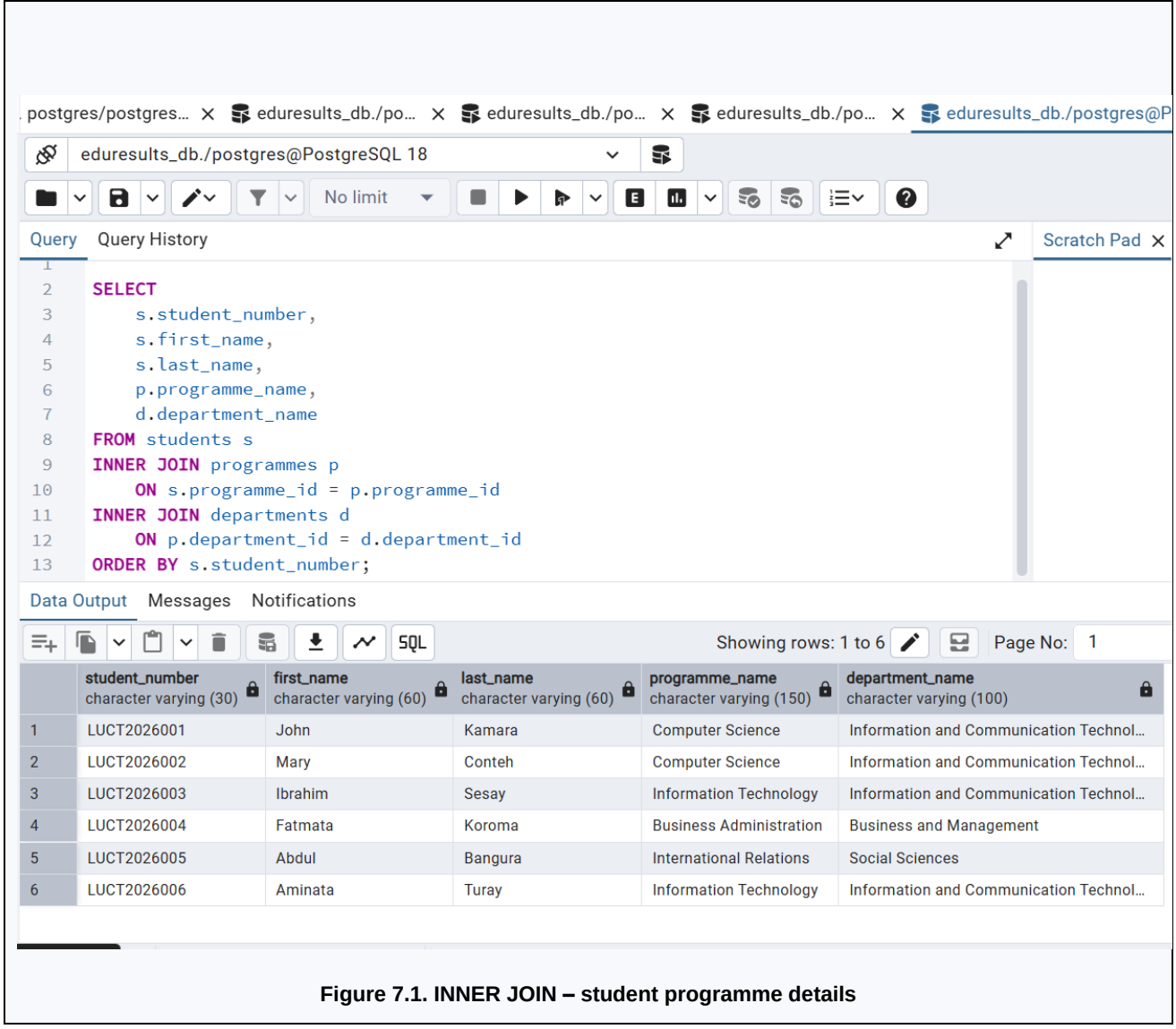


Figure 7.1. INNER JOIN – student programme details

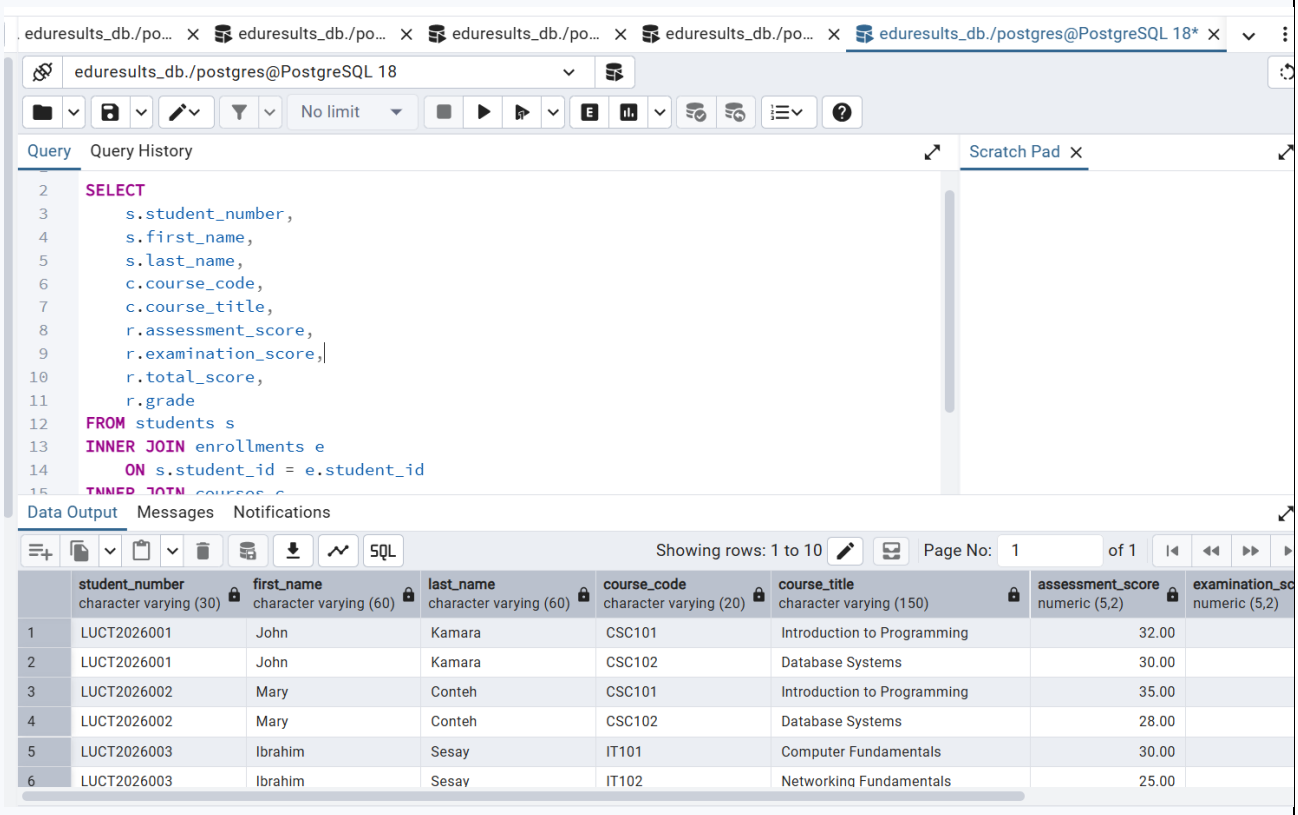


Figure 7.2. INNER JOIN – student examination results

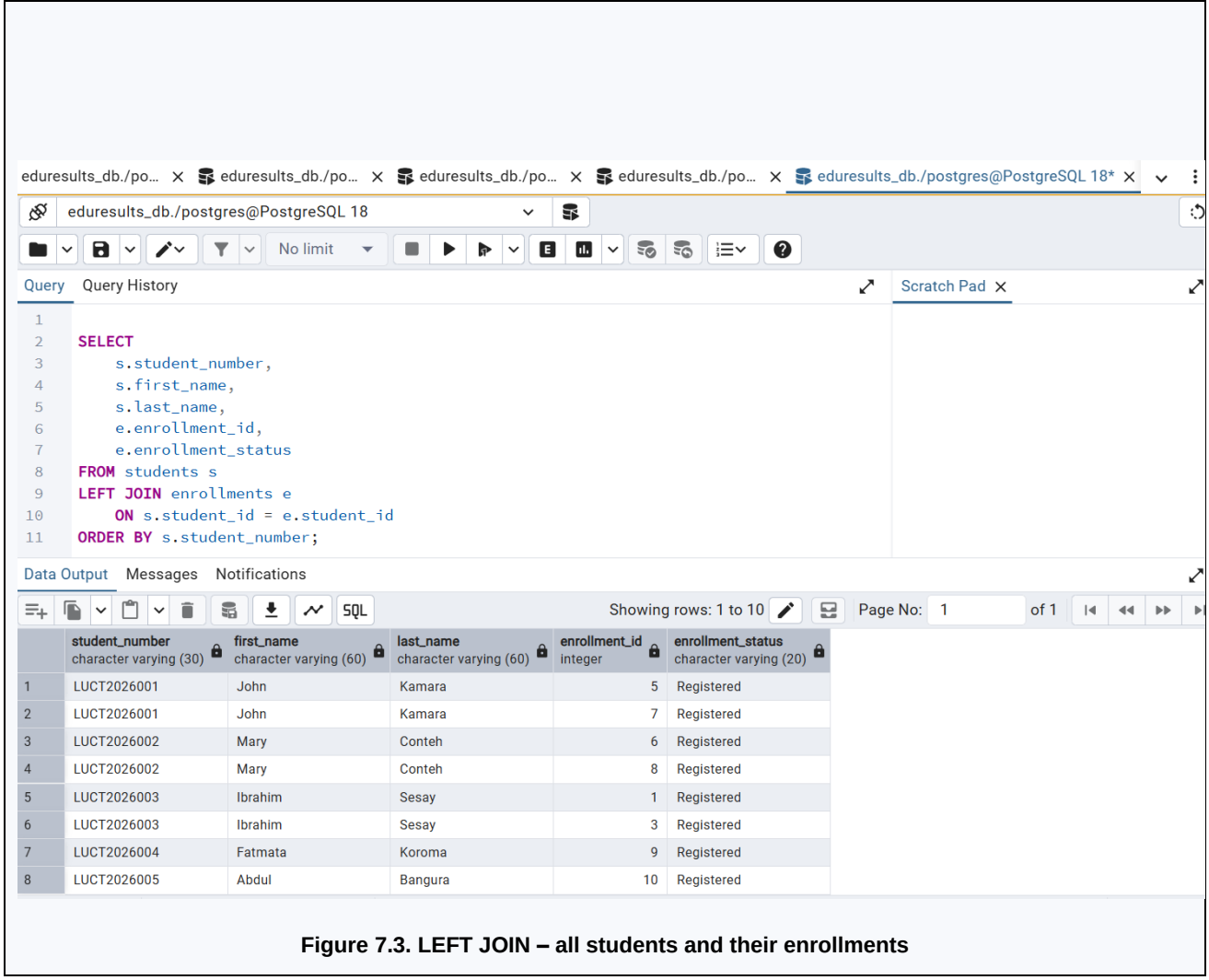


Figure 7.3. LEFT JOIN – all students and their enrollments

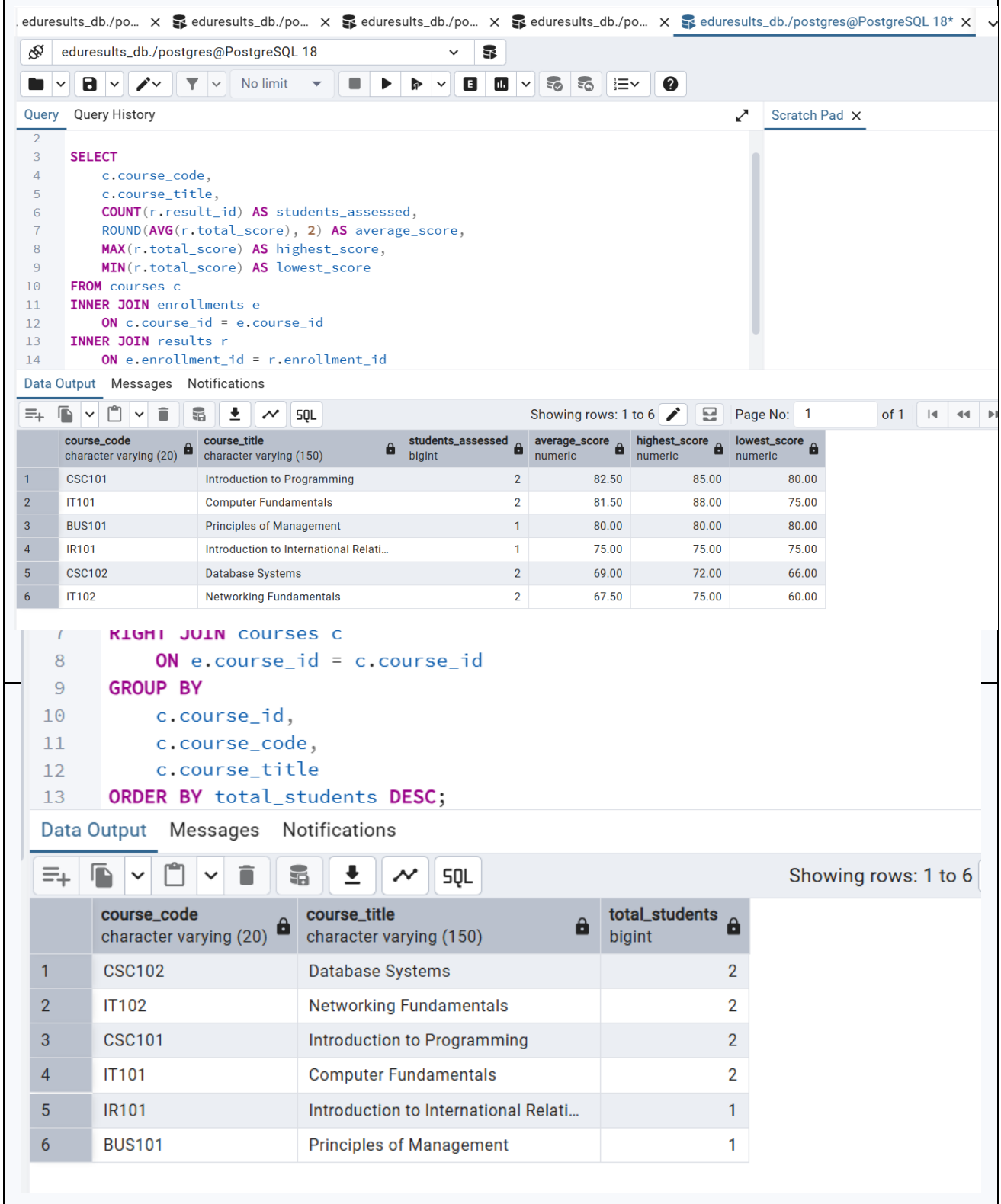
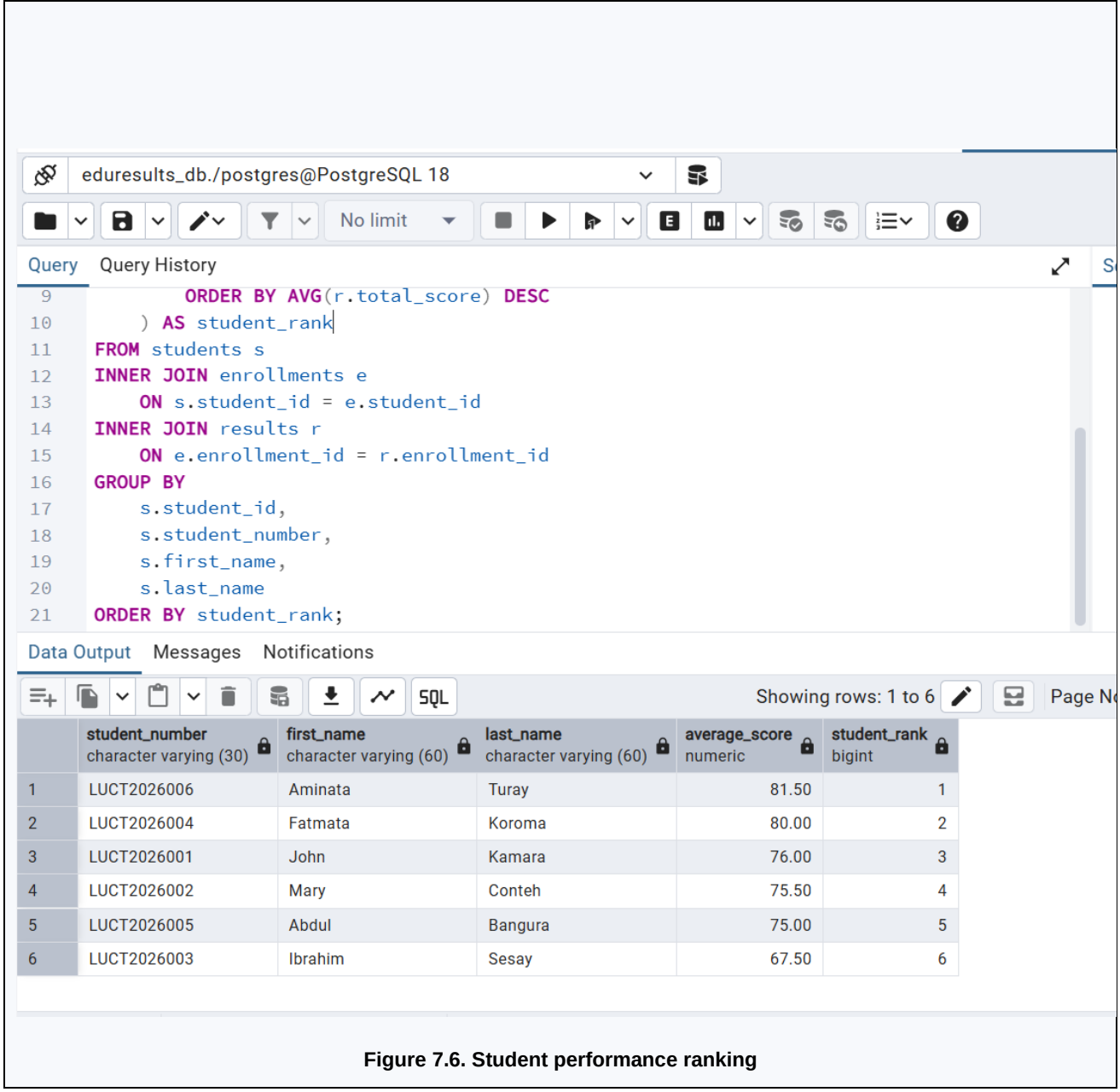


Figure 7.4. RIGHT JOIN – course enrollment summary



	student_number character varying (30)	first_name character varying (60)	last_name character varying (60)	average_score numeric	student_rank bigint
1	LUCT2026006	Aminata	Turay	81.50	1
2	LUCT2026004	Fatmata	Koroma	80.00	2
3	LUCT2026001	John	Kamara	76.00	3
4	LUCT2026002	Mary	Conteh	75.50	4
5	LUCT2026005	Abdul	Bangura	75.00	5
6	LUCT2026003	Ibrahim	Sesay	67.50	6

Figure 7.6. Student performance ranking

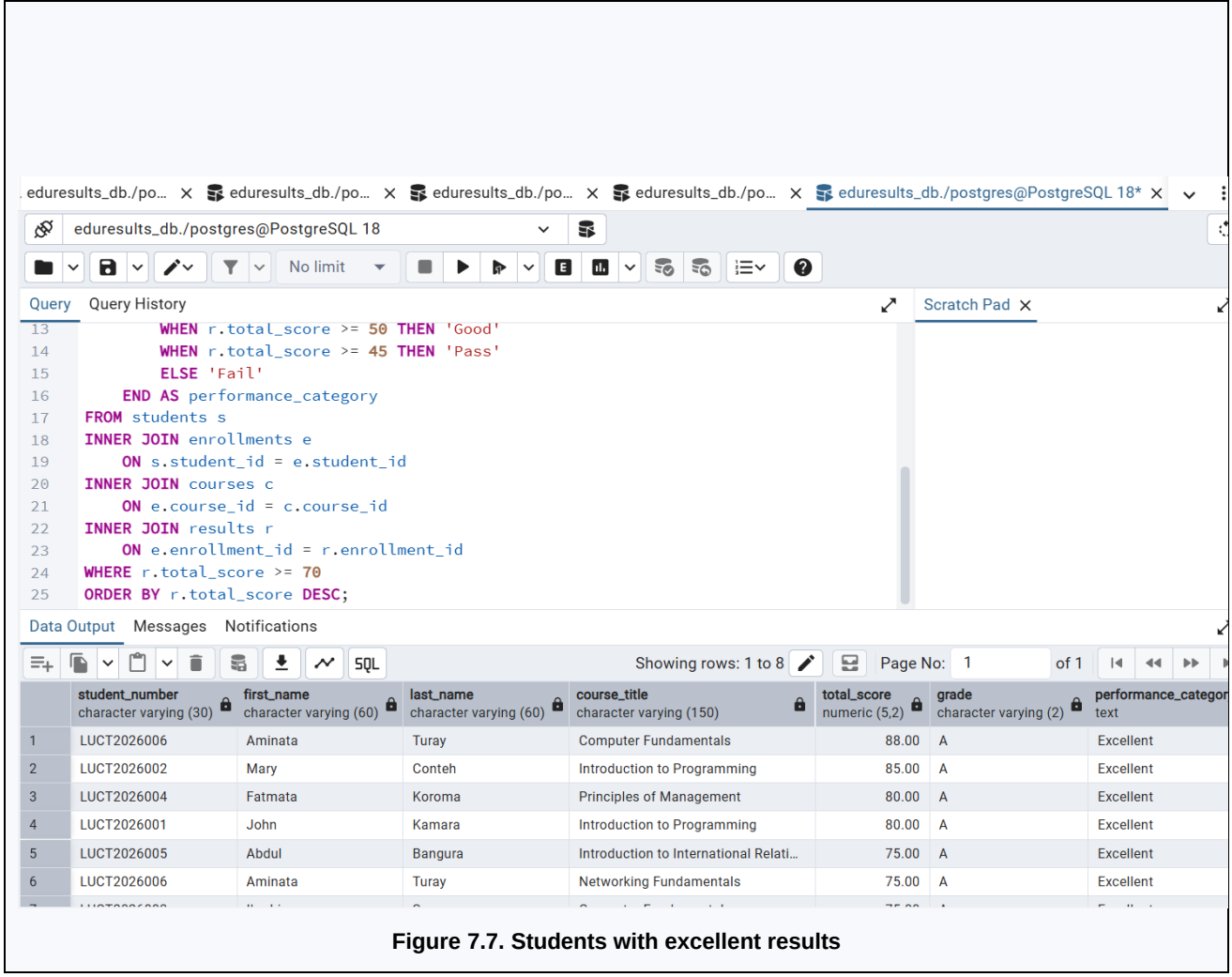


Figure 7.7. Students with excellent results

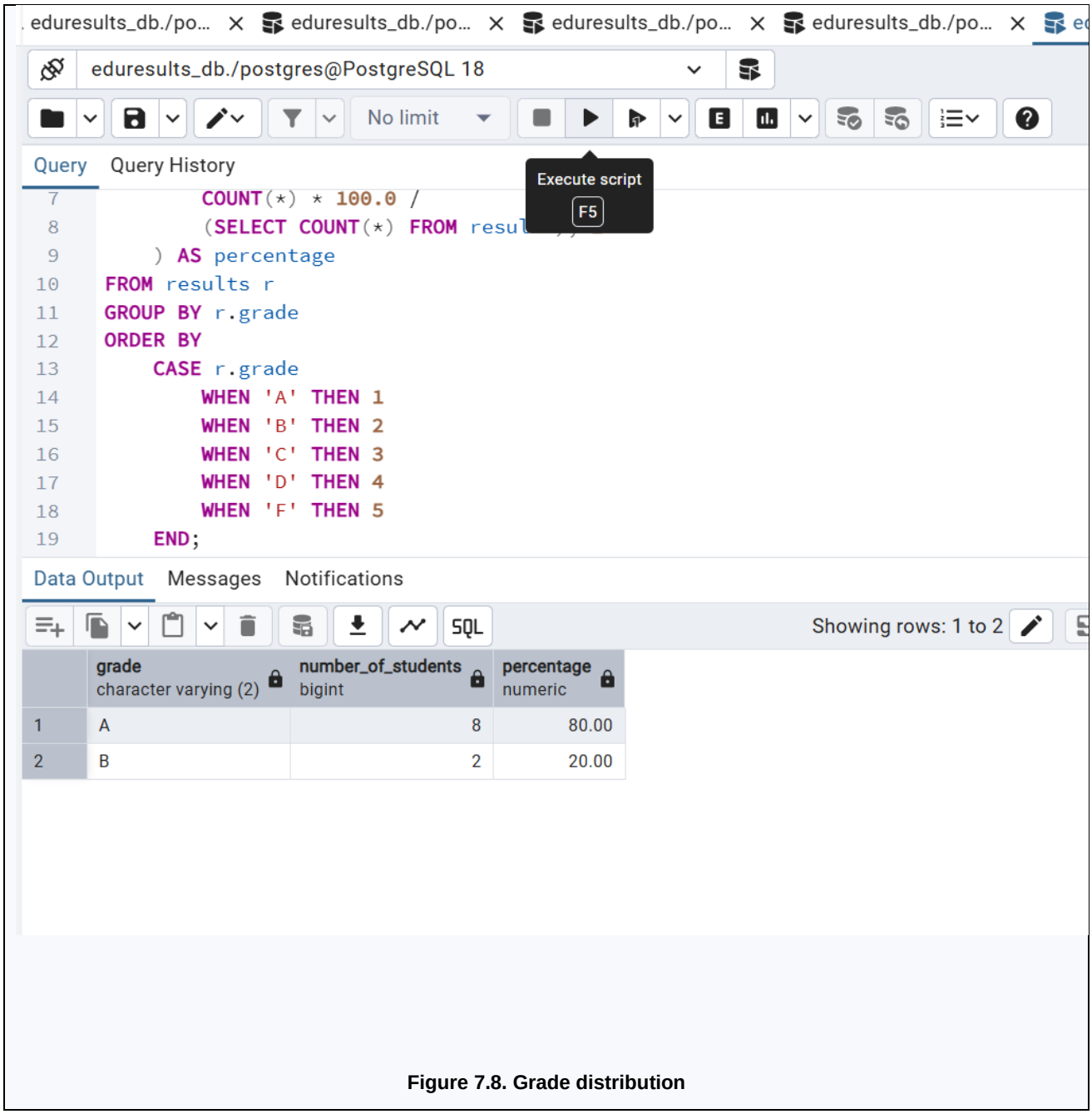


Figure 7.8. Grade distribution

8. POSTGRESQL VIEWS

A View is a named, reusable query that presents data from one or more underlying tables. A standard PostgreSQL View stores the query definition rather than maintaining a separate copy of the underlying rows. Users can query a View in much the same way as a table. EduResults contains five Views.

View name	Purpose
student_results_summary	Combines student, programme, enrollment, course and result information for an individual examination summary.
course_performance_summary	Summarises the number of results, average, highest and lowest scores, and grade counts for each course.
student_enrollment_summary	Presents student programme, course, academic-session and enrollment-status information.
lecturer_course_assignment	Presents lecturer, department, course and programme assignment information.
academic_grade_distribution	Summarises the number and percentage of results in each grade category.

The Views support routine reporting and reduce the need to repeatedly write the same multi-table queries. They do not replace the underlying tables; they present information derived from those tables.

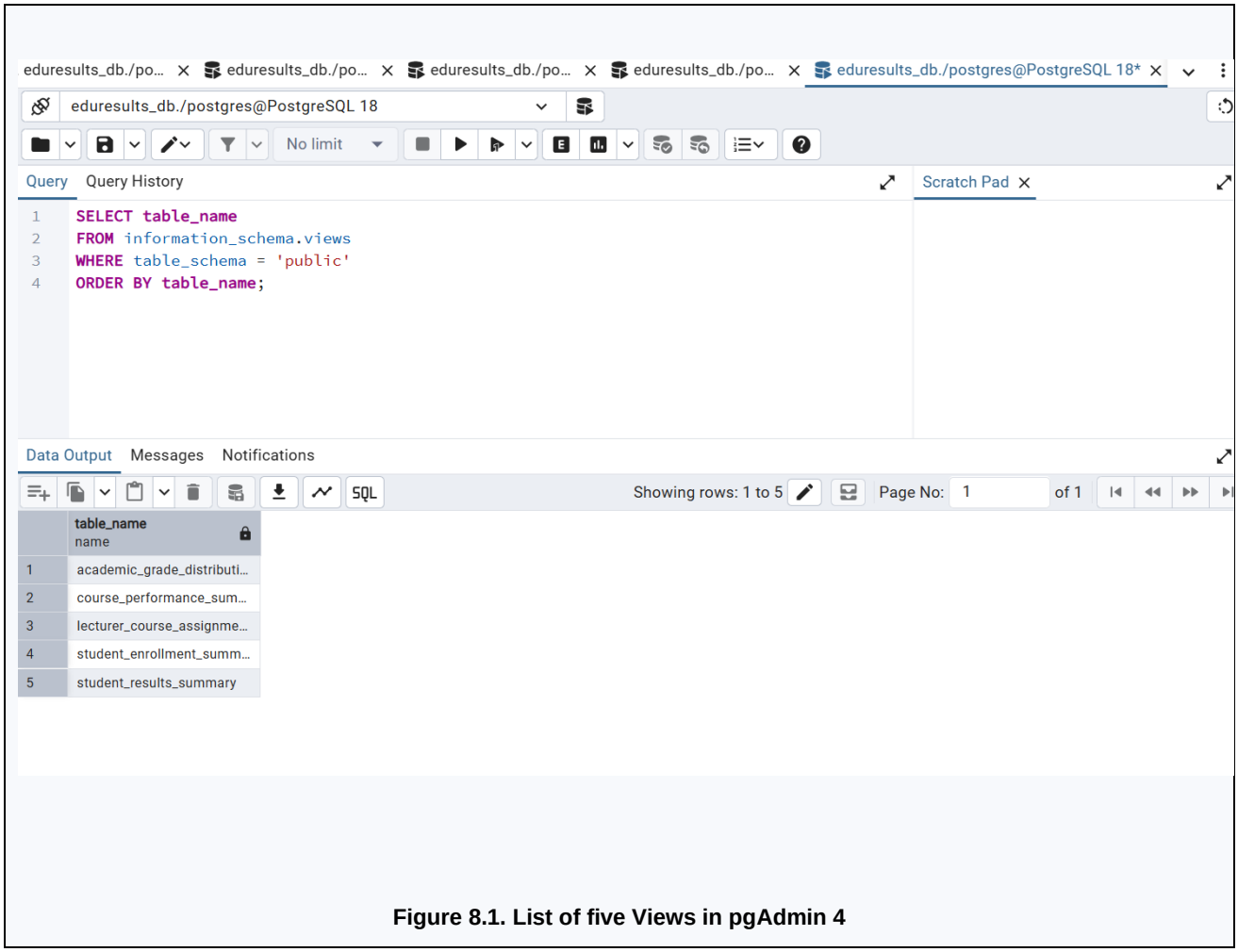


Figure 8.1. List of five Views in pgAdmin 4

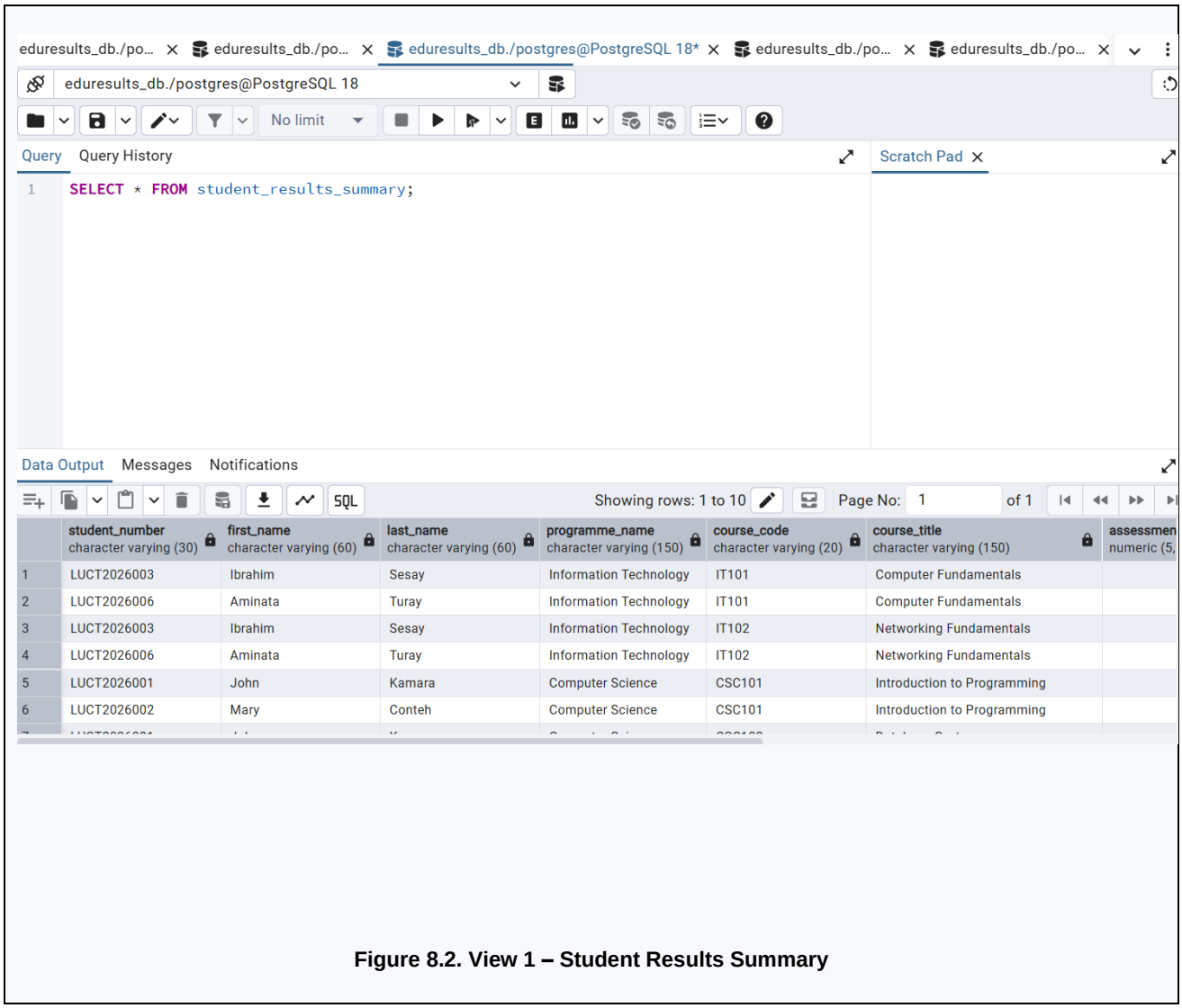
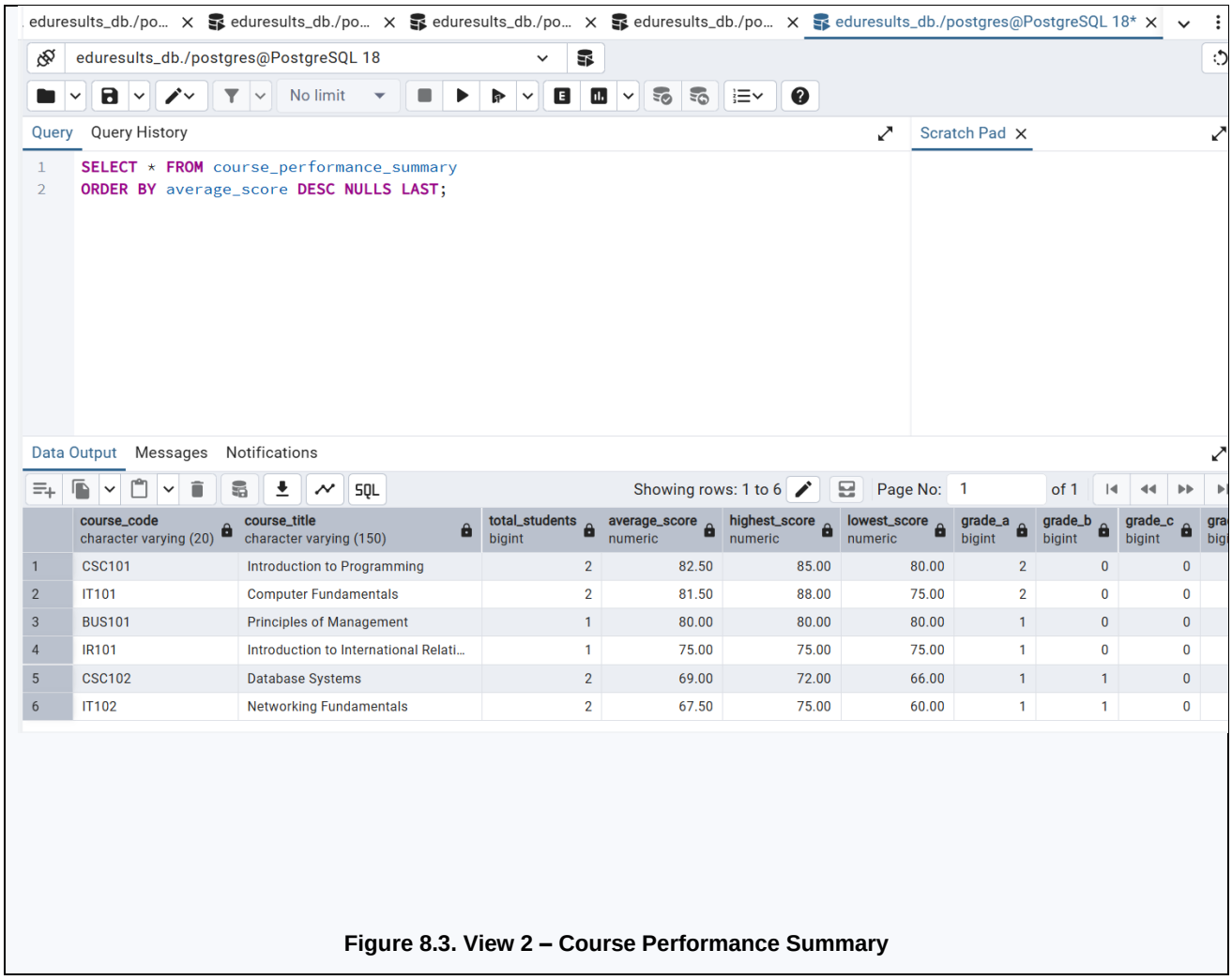
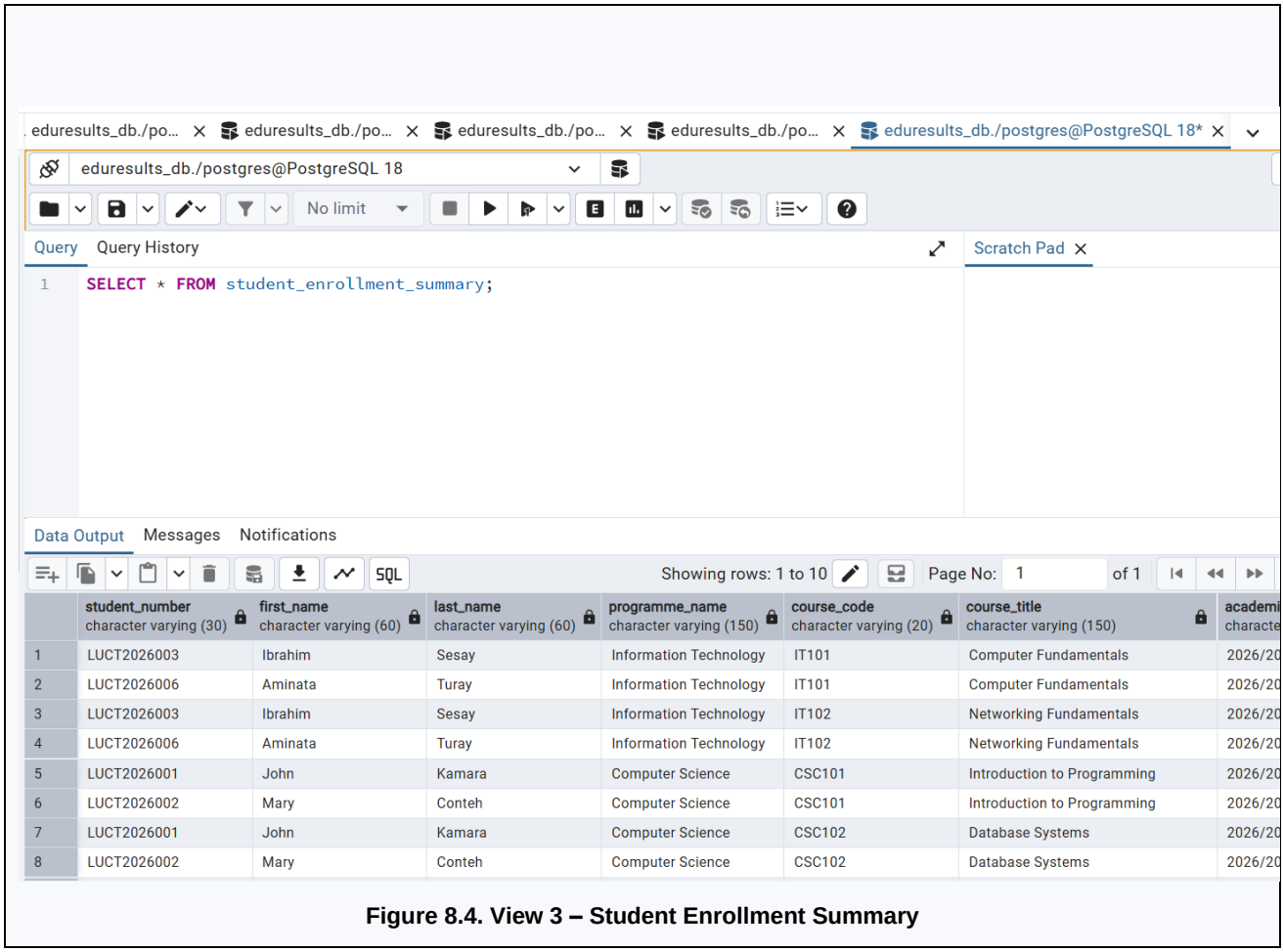


Figure 8.2. View 1 – Student Results Summary



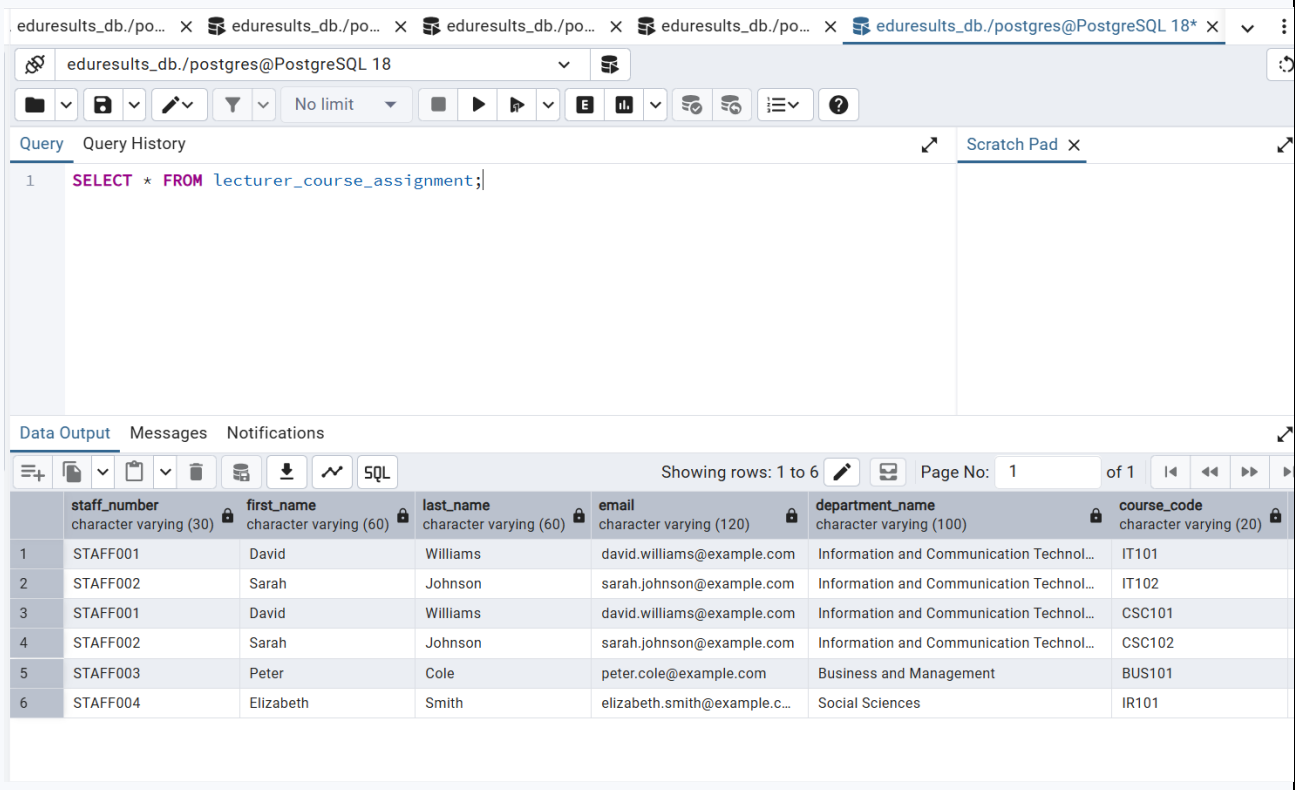
	course_code character varying (20)	course_title character varying (150)	total_students bigint	average_score numeric	highest_score numeric	lowest_score numeric	grade_a bigint	grade_b bigint	grade_c bigint
1	CSC101	Introduction to Programming	2	82.50	85.00	80.00	2	0	0
2	IT101	Computer Fundamentals	2	81.50	88.00	75.00	2	0	0
3	BUS101	Principles of Management	1	80.00	80.00	80.00	1	0	0
4	IR101	Introduction to International Relati...	1	75.00	75.00	75.00	1	0	0
5	CSC102	Database Systems	2	69.00	72.00	66.00	1	1	0
6	IT102	Networking Fundamentals	2	67.50	75.00	60.00	1	1	0

Figure 8.3. View 2 – Course Performance Summary



	student_number character varying (30)	first_name character varying (60)	last_name character varying (60)	programme_name character varying (150)	course_code character varying (20)	course_title character varying (150)	academi characte
1	LUCT2026003	Ibrahim	Sesay	Information Technology	IT101	Computer Fundamentals	2026/20
2	LUCT2026006	Aminata	Turay	Information Technology	IT101	Computer Fundamentals	2026/20
3	LUCT2026003	Ibrahim	Sesay	Information Technology	IT102	Networking Fundamentals	2026/20
4	LUCT2026006	Aminata	Turay	Information Technology	IT102	Networking Fundamentals	2026/20
5	LUCT2026001	John	Kamara	Computer Science	CSC101	Introduction to Programming	2026/20
6	LUCT2026002	Mary	Conteh	Computer Science	CSC101	Introduction to Programming	2026/20
7	LUCT2026001	John	Kamara	Computer Science	CSC102	Database Systems	2026/20
8	LUCT2026002	Mary	Conteh	Computer Science	CSC102	Database Systems	2026/20

Figure 8.4. View 3 – Student Enrollment Summary



	staff_number character varying (30)	first_name character varying (60)	last_name character varying (60)	email character varying (120)	department_name character varying (100)	course_code character varying (20)
1	STAFF001	David	Williams	david.williams@example.com	Information and Communication Technol...	IT101
2	STAFF002	Sarah	Johnson	sarah.johnson@example.com	Information and Communication Technol...	IT102
3	STAFF001	David	Williams	david.williams@example.com	Information and Communication Technol...	CSC101
4	STAFF002	Sarah	Johnson	sarah.johnson@example.com	Information and Communication Technol...	CSC102
5	STAFF003	Peter	Cole	peter.cole@example.com	Business and Management	BUS101
6	STAFF004	Elizabeth	Smith	elizabeth.smith@example.c...	Social Sciences	IR101

Figure 8.5. View 4 – Lecturer Course Assignment

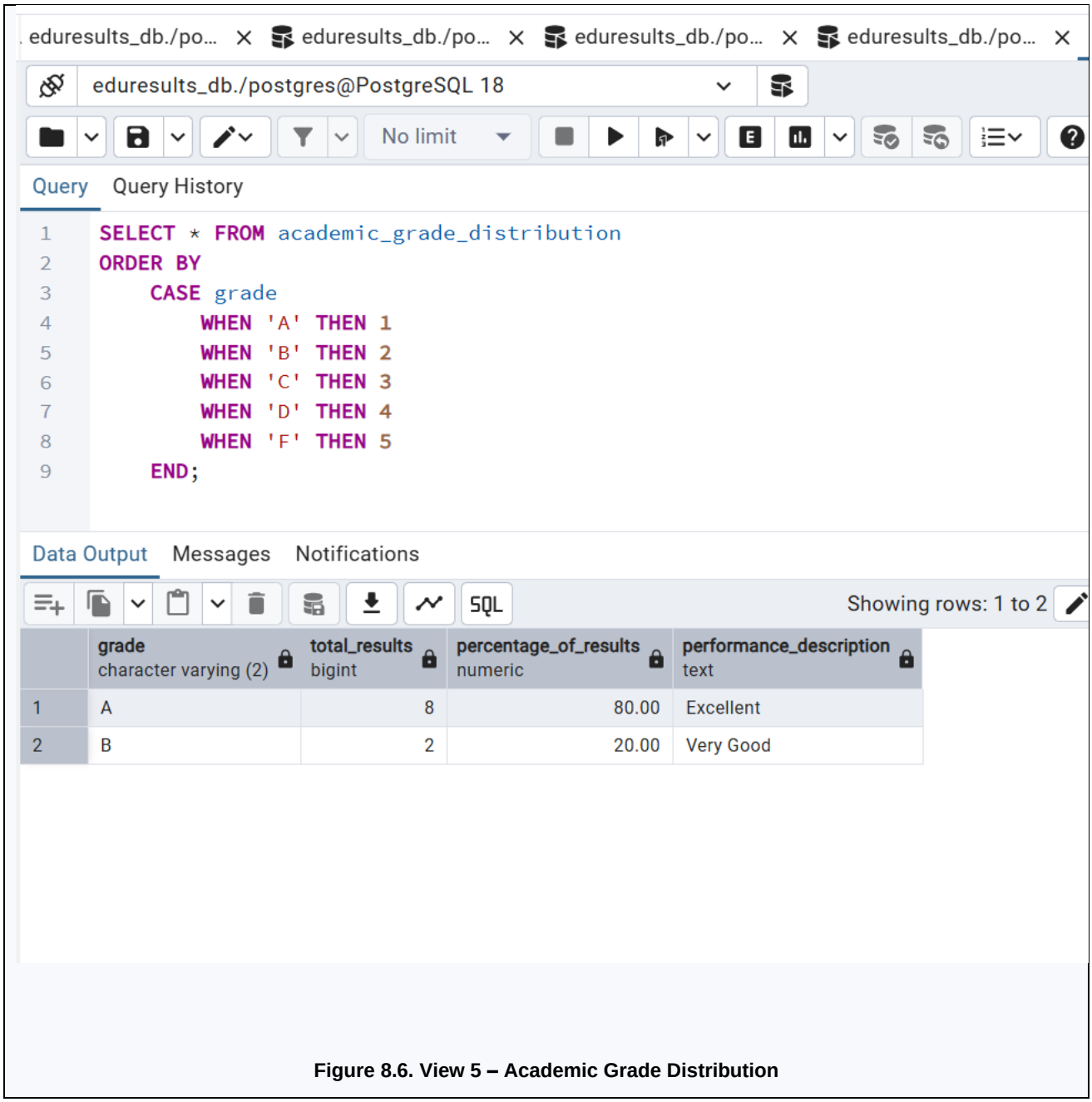


Figure 8.6. View 5 – Academic Grade Distribution

9. POSTGRESQL TRIGGERS

A Trigger is a database object that automatically invokes a Trigger Function when a specified event occurs on a table. EduResults implements two Triggers to demonstrate automation and accountability.

9.1 Trigger One – Automatic enrollment status update

The first Trigger is attached to the results table. After a result row is inserted or its result_status is updated, the Trigger Function checks whether the new result status is Published. If it is, the corresponding enrollment record is updated to Completed. This reduces the need for a separate manual status update.

The test demonstrated a result with result_id 1 and result_status Published, while its corresponding enrollment record showed enrollment_status Completed. This output is evidence that the Trigger operated as designed for the tested record.

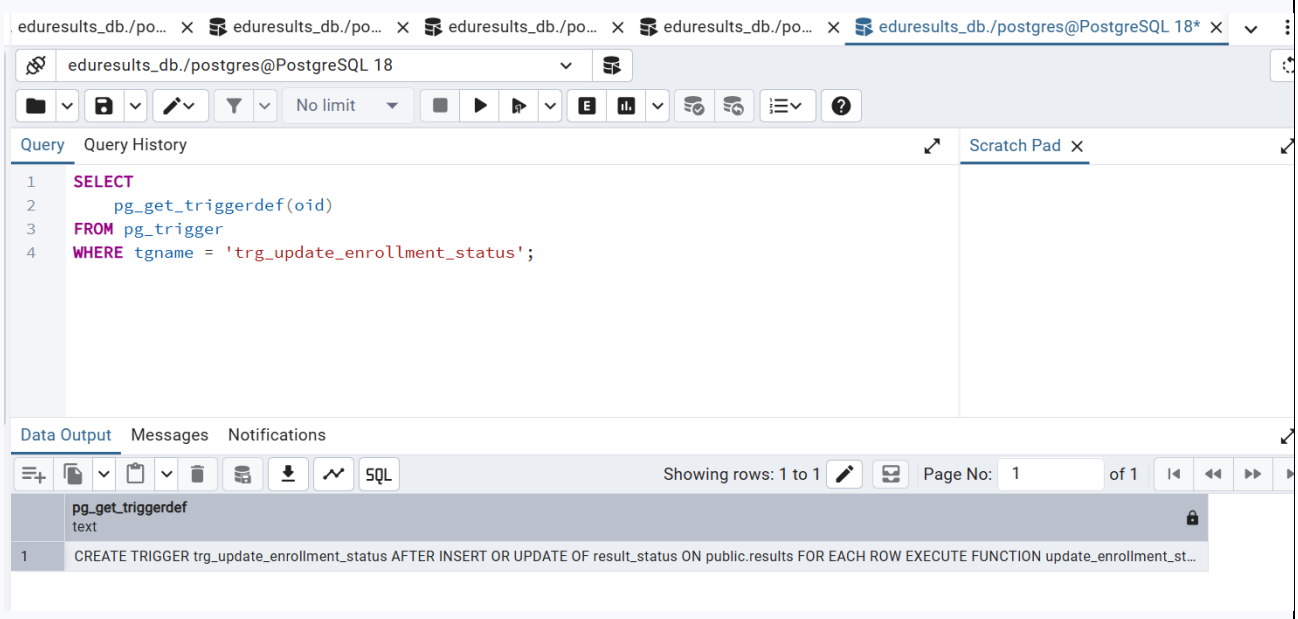


Figure 9.1. Trigger One – enrollment status update SQL or object

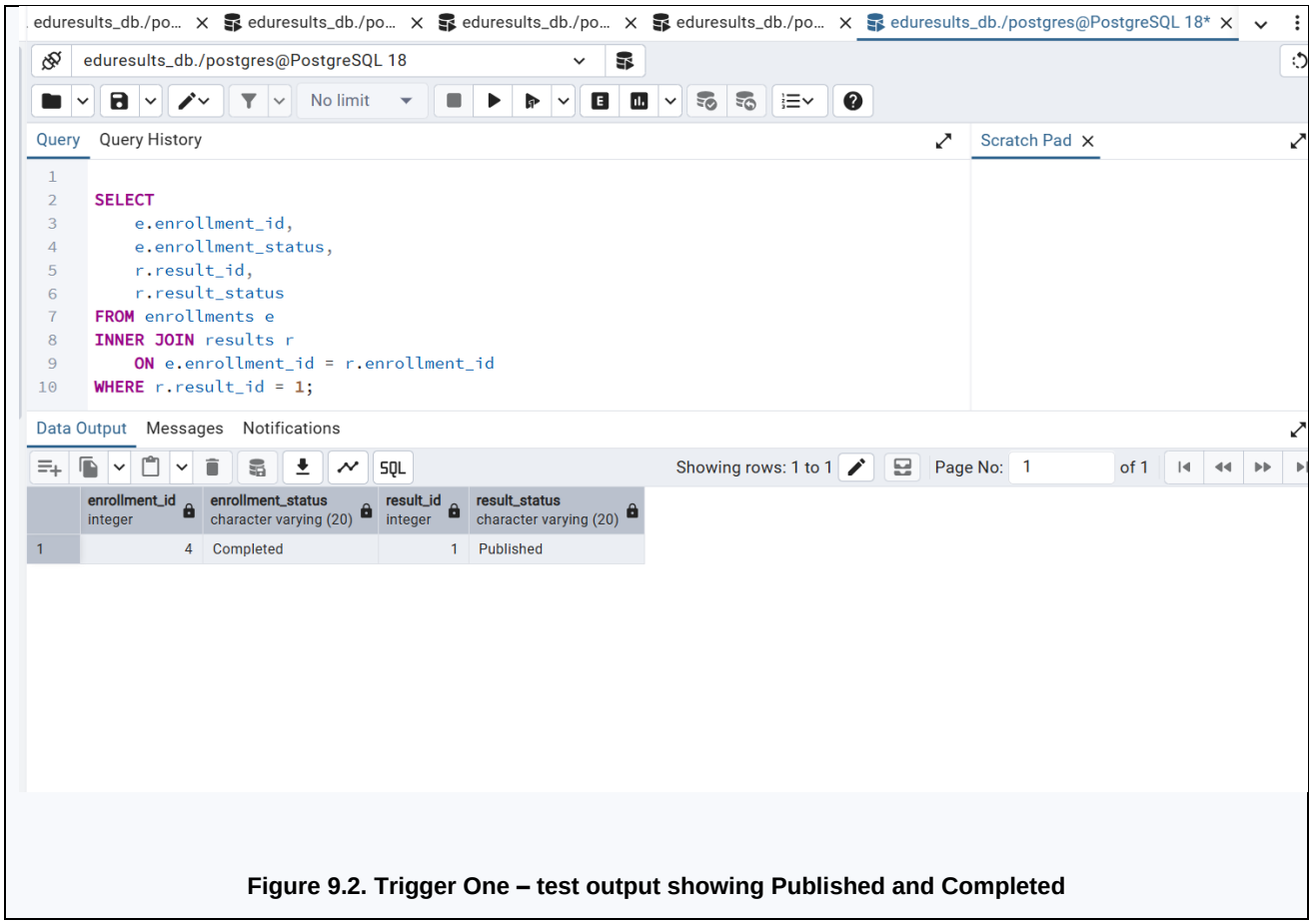


Figure 9.2. Trigger One – test output showing Published and Completed

9.2 Trigger Two – Examination result audit trail

The second Trigger is attached to the results table and runs after INSERT or UPDATE operations. Its Trigger Function writes an audit row to result_audit. For updates, the audit record captures old and new assessment scores, old and new examination scores, and old and new grades, together with the result ID, operation type, timestamp and database user.

During testing, the assessment score for result_id 1 was changed from 31.00 to 33.00. The audit table recorded the UPDATE operation and the old and new values. This demonstrates how the audit Trigger can support traceability of examination-result changes.

The current audit Trigger records INSERT and UPDATE operations. It does not record DELETE operations. The audit table is a separate supporting table and may be empty before the first audited operation.

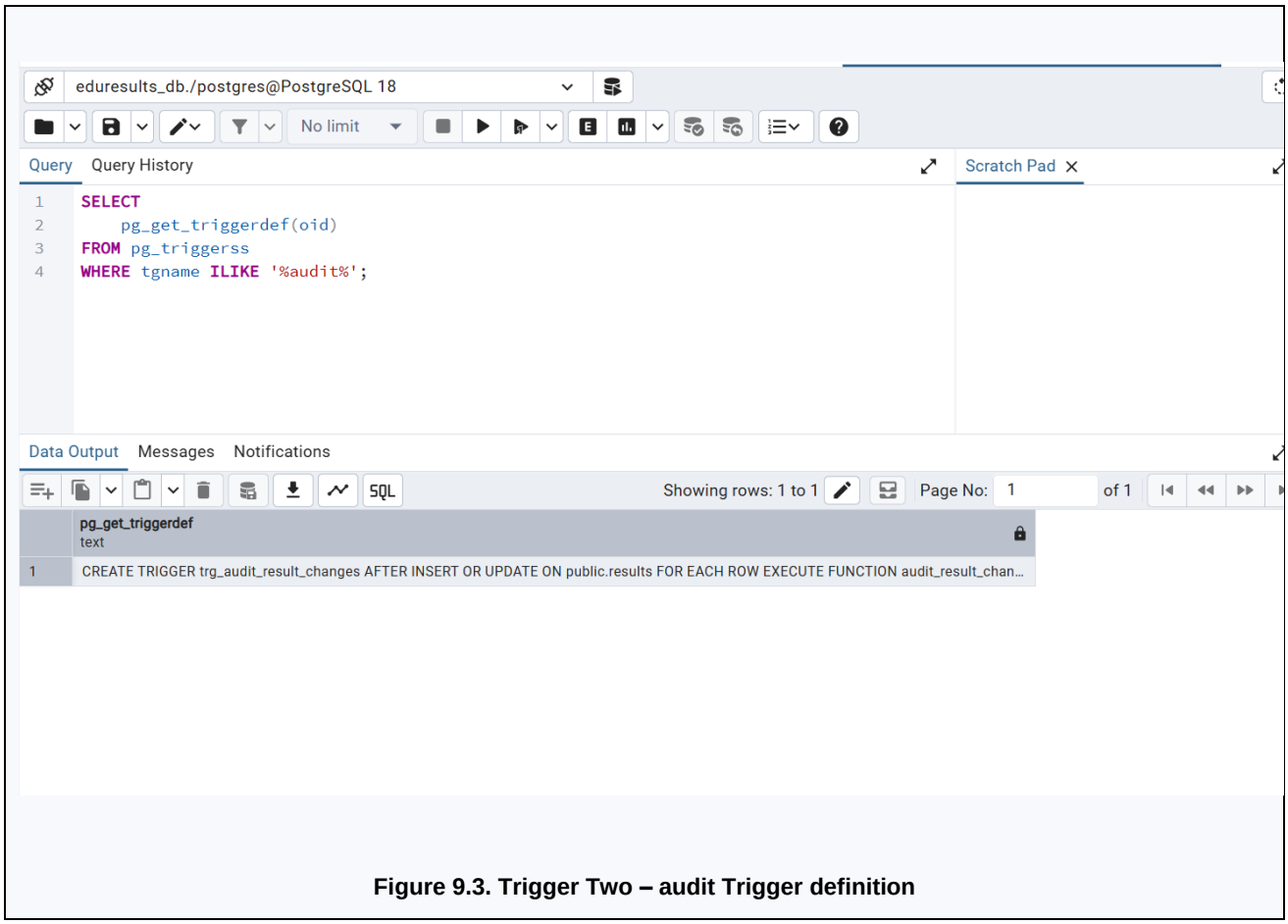


Figure 9.3. Trigger Two – audit Trigger definition

10. STORED PROCEDURE

A Stored Procedure is a named set of procedural instructions stored in the database and invoked when required. EduResults includes the procedure register_student_for_course. It accepts a student number, course code, academic year and semester as input parameters.

10.1 Procedure operation

1. Finds the student record using the supplied student number.
2. Finds the course using the supplied course code.
3. Finds the academic session using the academic year and semester.
4. Raises an exception if the supplied student, course or session cannot be found.
5. Attempts to insert an enrollment with the Registered status.
6. Uses the configured conflict handling to avoid inserting a duplicate student-course-session combination.

10.2 Procedure testing

The procedure was called with student number LUCT2026007, course code CSC101, academic year 2026/2027 and semester 1. PostgreSQL displayed the notice 'Student registration processed successfully'. A subsequent verification query joined enrollments, students, courses and academic_sessions. The output showed Mohamed Jalloh registered for CSC101 (Introduction to Programming) in the 2026/2027 academic year, Semester 1, with enrollment status Registered.

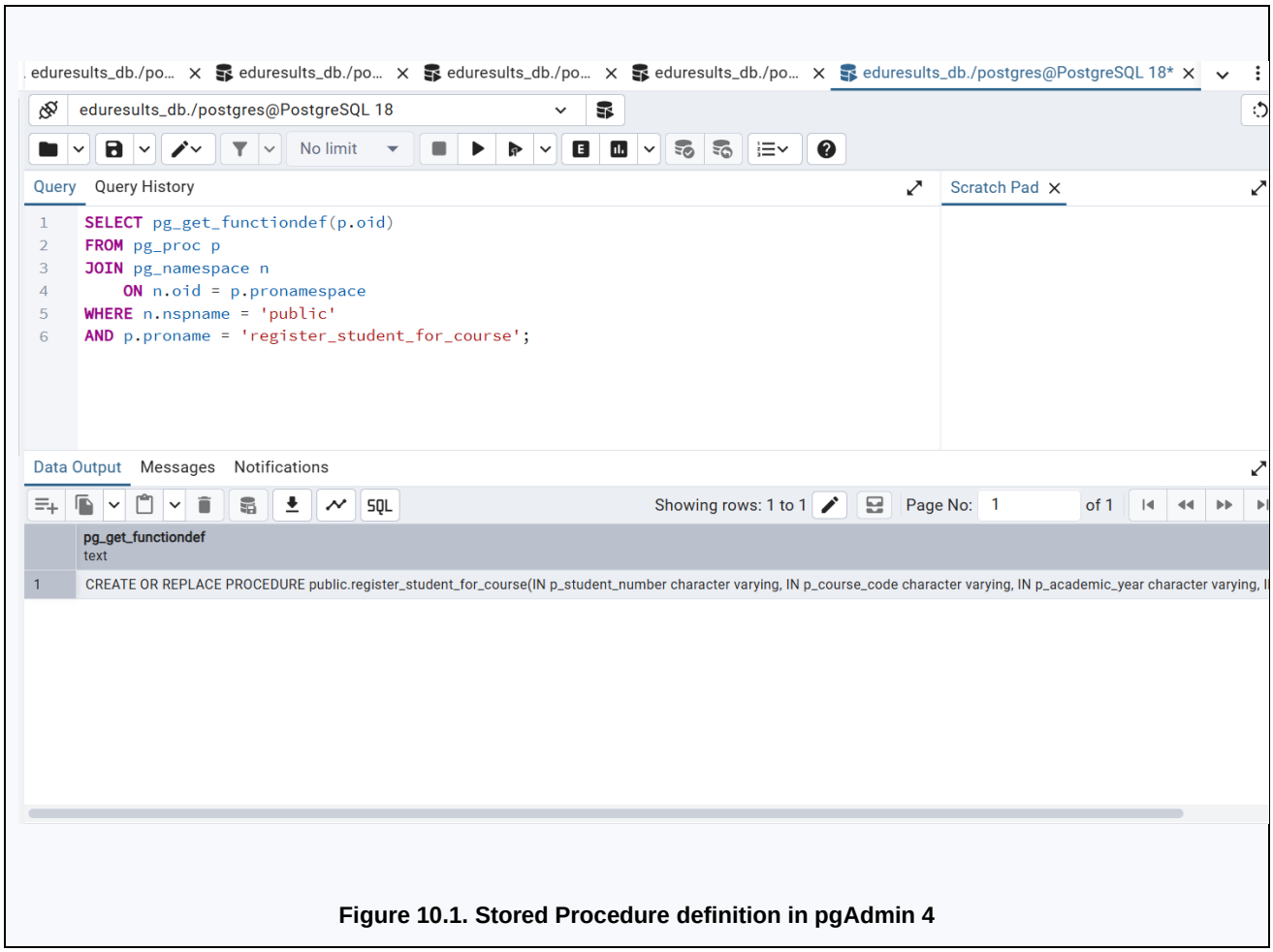


Figure 10.1. Stored Procedure definition in pgAdmin 4

11. TESTING AND VERIFICATION

Testing was carried out by executing SQL statements in pgAdmin 4 and reviewing the Messages and Data Output panels. The project work included checking that the database objects were created, that the reporting queries returned data, and that the automation and registration components performed their intended operations.

Component	Test performed	Observed evidence
Tables	Inspected the public schema table list.	Eight core tables plus the result_audit supporting table were created.
JOIN queries	Executed INNER, LEFT and RIGHT JOIN queries and analytical queries.	Query output displayed combined and summarised records.
Views	Queried the information_schema view list and ran the individual Views.	Five named Views were present and used for reporting.
Trigger One	Published a result and checked the related enrollment.	The tested enrollment status appeared as Completed.
Trigger Two	Updated an assessment score and queried result_audit.	The audit row recorded the old and new assessment scores.
Stored Procedure	Called register_student_for_course and queried the enrollment.	Mohamed Jalloh's CSC101 enrollment appeared with Registered status.

The tests described above reflect the operations demonstrated during the project work. Before submission, the student should ensure that all screenshots are readable and that the final SQL script matches the database objects currently installed in PostgreSQL.

12. SECURITY, DATA INTEGRITY AND LIMITATIONS

12.1 Data integrity

Primary keys provide unique identifiers for records, while foreign keys help prevent references to missing parent records. The Stored Procedure checks that the requested student, course and academic session exist before attempting registration. Its duplicate-conflict handling is intended to prevent repeated registration for the same student, course and session, provided the relevant unique constraint exists on the enrollment table.

12.2 Auditability

The result_audit table provides a basic record of result insertions and updates. In a production deployment, access to the audit table should be restricted, backups should be scheduled, and audit records should be protected from unauthorised modification or deletion.

12.3 Limitations and future improvements

- Add role-based access control for administrators, lecturers, examination officers and students.
- Add authentication and password-management facilities if a user-facing application is developed.
- Extend the audit Trigger to record DELETE operations where required by institutional policy.
- Add validation rules for score ranges, grading rules and permitted result-status transitions.
- Add transcript generation, result publication controls and reporting exports.
- Connect the database to a web or desktop application with appropriate security controls.
- Implement automated backups, recovery procedures and routine database maintenance.

13. CONCLUSION AND RECOMMENDATIONS

The EduResults project demonstrates the design and implementation of a relational database for managing student and academic information. Using PostgreSQL 18 and pgAdmin 4, the project created the core academic tables, inserted sample records, retrieved related data through SQL JOINs, and developed five Views for academic reporting. It also implemented two Triggers to automate enrollment-status updates and maintain an audit trail of result changes, together with a Stored Procedure for registering students for courses.

The successful tests of the two Triggers and the registration procedure demonstrate the practical value of database automation. The project provides a foundation that could be extended into a more complete student information system. Future development should prioritise user access controls, stronger validation, backup and recovery arrangements, and integration with a secure application interface.

Beyond the technical implementation, the EduResults project has provided practical experience in understanding how different database components work together to support an organized information management system. The development process demonstrated the importance of establishing appropriate relationships between database tables, maintaining data integrity, and using SQL commands to retrieve meaningful information. The application of JOIN operations and Views also showed how complex academic information can be presented in a more organized and accessible manner.

Furthermore, the implementation of Triggers and Stored Procedures demonstrated the importance of database automation in reducing repetitive manual operations and improving the consistency of database activities. The enrollment-status update trigger provides an automated approach to updating enrollment records following result publication, while the examination-result audit trigger introduces a mechanism for monitoring changes to academic records. Similarly, the student registration procedure demonstrates how frequently performed database operations can be organized into reusable procedures. These features collectively strengthen the functionality and practical relevance of the EduResults database.

In conclusion, EduResults has achieved its primary objective of demonstrating the design and implementation of a functional relational database using PostgreSQL. It has also provided valuable practical knowledge and experience in database development, SQL programming, testing, backup management, and database administration. Although the current implementation focuses primarily on database operations rather than a complete user-facing application, it provides a foundation for further development into a comprehensive Student Examination and Result Management System. The successful completion of this project reflects the practical application of the database concepts and advanced PostgreSQL techniques covered in the Database Systems course.